School of CSE, REVA University

# Codebase-Wide Version Migration & Refactoring Agent

## Detailed Design Document

Module 2: Static Analysis and Call-Graph Mapping Module
Flowcharts, UML Diagrams and Data Flow Diagrams

Team: Adithya Naik (R23EF014), Prajwal Ragavendra Hawaldar (R23EF183), Verril Vaz (R23EN179), Nandan P (R23EH082), Prabanjan R (R23EF181)
Faculty Guide: Dr. Nimrita Koul, Center for Artificial Intelligence & Machine Learning
Problem Statement Identifier: CAIML_Nimrita_Koul_3_PS_209
October 2026

## 1. Introduction

### *1.1 Purpose*

This document gives the detailed design of Module 2, the Static Analysis and Call-Graph Mapping Module, of the migration agent. The system design document specified the module's place in the overall architecture and its interface contract. This document goes one level down: it describes the internal logic of the module as flowcharts, its structure and interactions as UML class and sequence diagrams, and its data movement as data flow diagrams at two levels of detail.

### *1.2 Why this module is detailed first*

Every later module depends on the output of Module 2. The planner orders files using its dependency graph, the graph memory is populated with its file nodes and call sites, the edit engine edits the sites it reports, and the benchmark harness divides by the baseline count it produces. An error here, such as a missed call site, cannot be recovered by the later modules because they never see the site. The module therefore has the strictest correctness requirement in the system, and the design below is organised around making each of its steps explicit and testable.

### *1.3 References*

[1] Literature Survey and Requirement Analysis, CAIML_Nimrita_Koul_3_PS_209, Task 1. [2] System Design Document, CAIML_Nimrita_Koul_3_PS_209, Task 2. [3] Team Overview, Module Description and Technology Stack, Internship Week 1.

## 2. Module Overview

Module 2 analyses the target repository before any edit is made. It parses every source file, builds the import graph, the symbol table and the call graph, finds every call site of a deprecated API, computes the set of functions and files affected through their callers, and produces an independent baseline count of deprecated usages. It satisfies functional requirements FR2 to FR5 of the requirement analysis.

| Direction | Item | Peer module | Content |
|---|---|---|---|
| Input | MigrationSpec | Module 1 | Deprecated patterns, each with an AST matcher and a replacement reference |
| Input | Repository path | Run configuration | Root of the source tree and exclude rules |
| Output | Dependency graph, impact set | Module 3 (planner) | File-level graph and the set of functions and files to change |
| Output | File nodes, call sites, unresolved sites, parse failures | Module 4 (memory) | Initial state for every file in the graph memory |
| Output | Baseline usage count | Module 9 (benchmark) | Number of deprecated usages before migration, from the oracle detector |

Table 1. Inputs and outputs of Module 2

### *2.1 Internal decomposition*

The module is divided into seven sub-components. Each corresponds to one process in the level 1 data flow diagram and to one class in the class diagram.

| ID | Sub-component | Class | Responsibility | FR |
|---|---|---|---|---|
| 2.1 | Repository scan | RepositoryScanner | Collect Python source files and apply exclude rules | FR2 |
| 2.2 | Source parsing | SourceParser | Parse each file; use a fallback parser on failure; record failures without aborting | FR2 |
| 2.3 | Indexing | ImportGraphBuilder, SymbolTable | Build the file dependency graph and the symbol table that resolves aliases | FR3 |
| 2.4 | Call-site matching | CallSiteMatcher | Find call sites matching deprecated patterns and report unresolved ones | FR4 |
| 2.5 | Call graph | CallGraphBuilder | Build caller-to-callee edges between functions | FR4 |
| 2.6 | Impact analysis | ImpactAnalyzer | Find transitive callers affected by the call sites | FR5 |
| 2.7 | Oracle | OracleDetector | Count deprecated usages independently of 2.4 | FR14 (metric) |

Table 2. Sub-components of Module 2

## 3. Flowcharts

Four flowcharts describe the module. Figure 1 shows the complete run. Figures 2 to 4 expand the three subprocesses that contain loops or non-trivial decisions: scanning and parsing, call-site resolution, and impact-set computation.

### *3.1 Overall analysis flow*

The module runs its steps in a fixed order because each step consumes the output of the one before it: the symbol table needs the parsed modules, and call-site matching needs the symbol table. The only early exit is the case where no file could be parsed, which is reported as an error because every later step would operate on an empty repository.

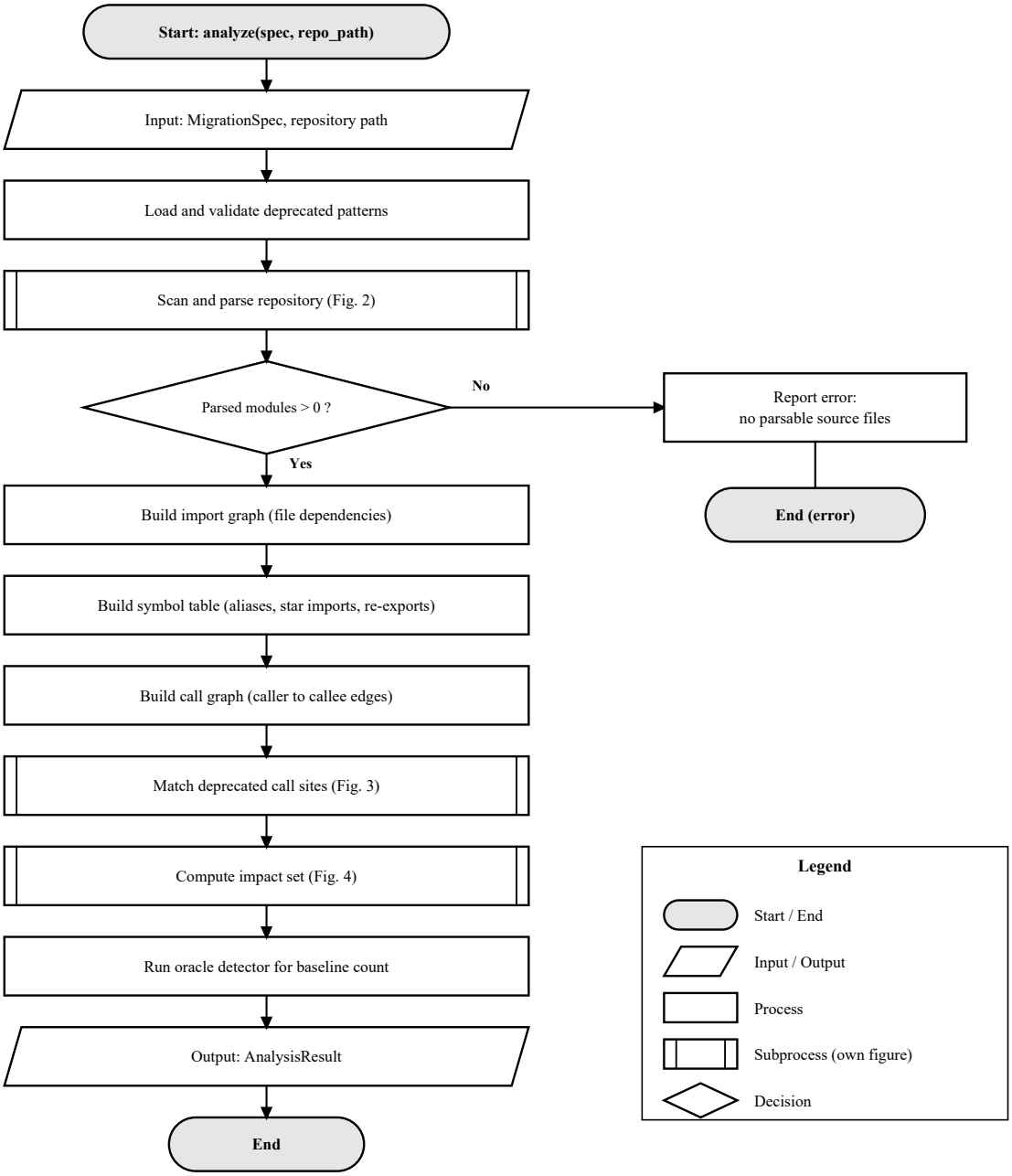



Fig. 1. Flowchart of the complete analyze() operation.

### *3.2 Scanning and parsing*

A syntax error in one file must not stop the analysis, so a parse failure is handled in two stages. The primary parser is tried first. If it fails, a fallback parser is tried, which is also how Python 2 sources are read, since the Python 3 ast module cannot parse them. Only if the fallback also fails is the file recorded as a ParseFailure and skipped. The failure list is returned to the graph memory so that the file is visible in the final report rather than silently missing.

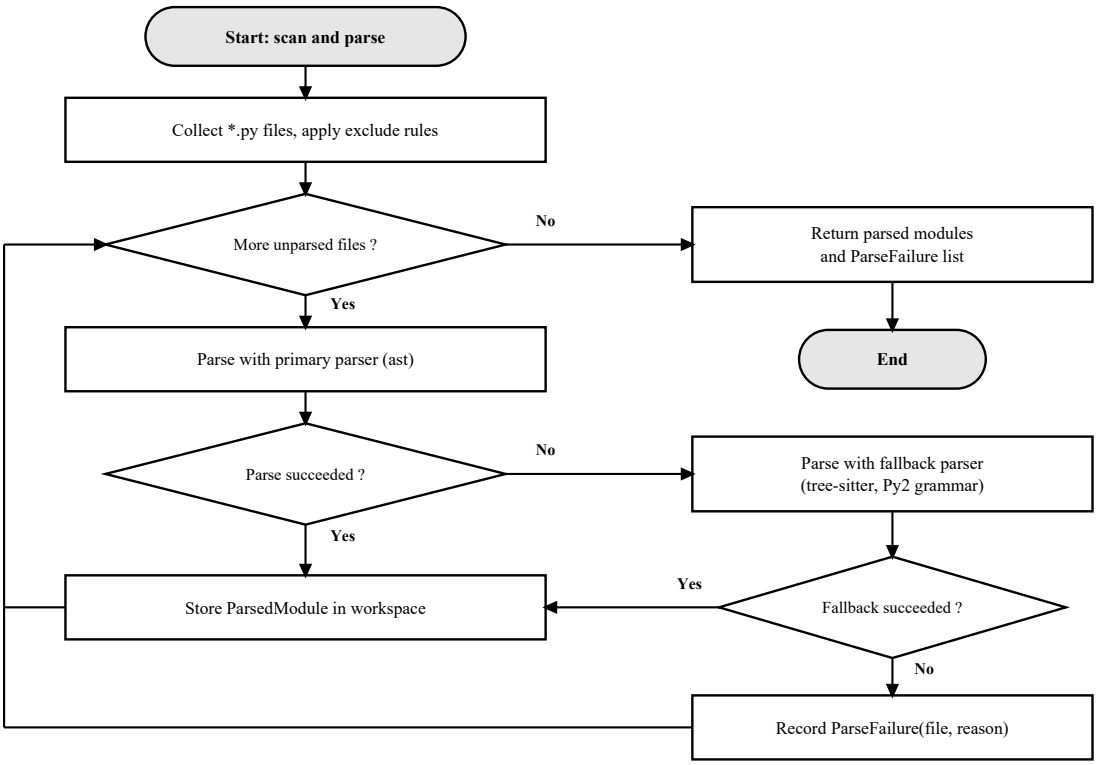



Fig. 2. Flowchart of the scan and parse subprocess.

### *3.3 Call-site resolution*

This is the core decision logic of the module. For each call expression the receiver is resolved to a qualified name through the symbol table. A resolved name is compared with the deprecated patterns; a match produces a CallSite. A name that cannot be resolved is not simply discarded: if the expression uses dynamic dispatch such as getattr or eval it is recorded as an UnresolvedSite with a reason, so that Module 7 can later use the test results to find it. Unresolved names that are plain external calls are ignored.

### *3.4 Impact-set computation*

The impact set is found by a breadth-first walk up the call graph. It starts from the functions that contain a deprecated call site and visits their callers, then their callers' callers, up to a configured depth limit. The visited set prevents revisiting a function that is reachable by several paths, which also makes the walk terminate on recursive call cycles.

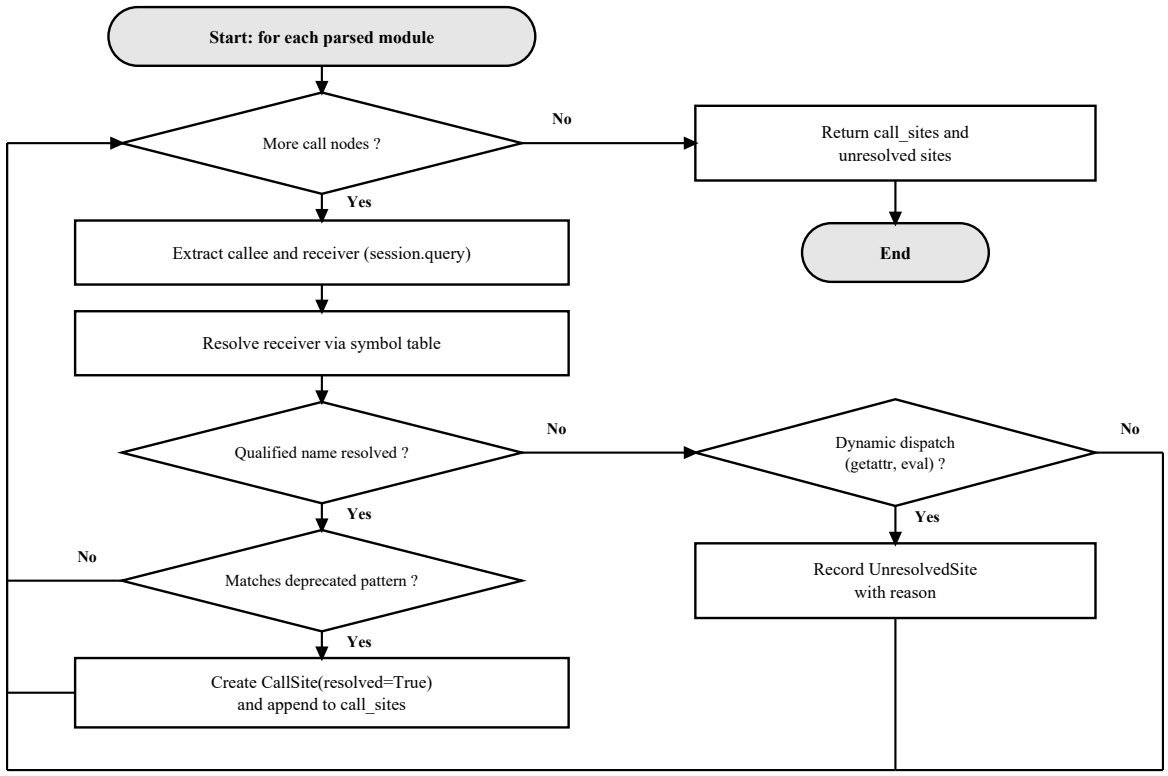



Fig. 3. Flowchart of call-site resolution for each call node.

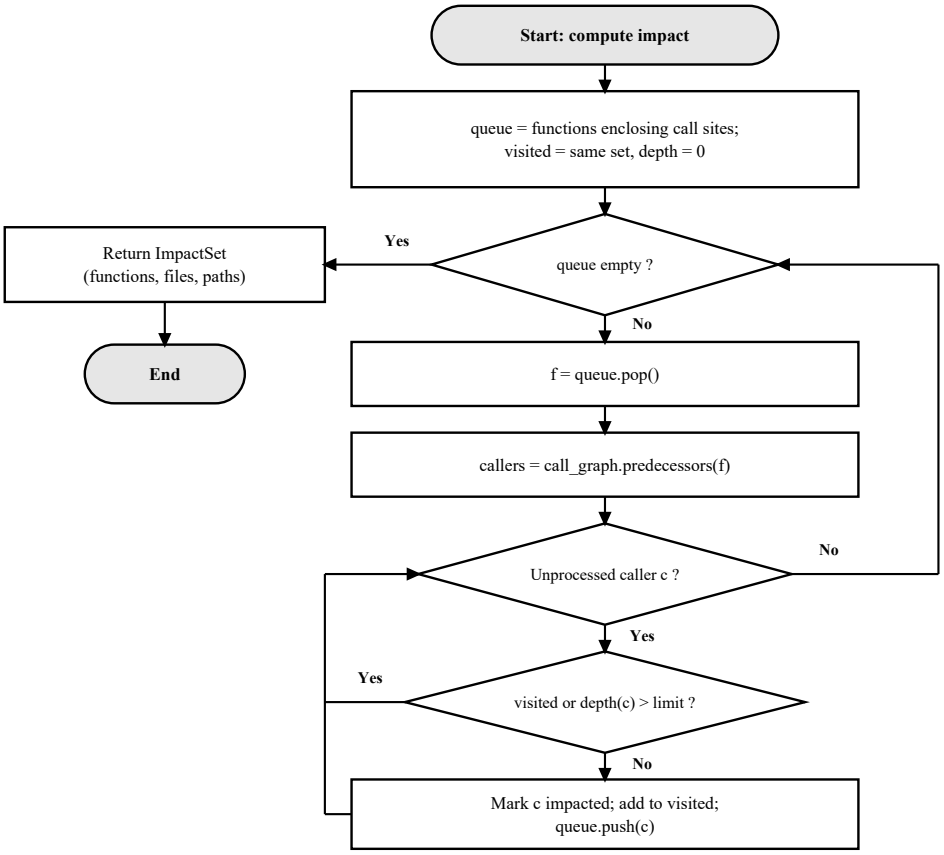



Fig. 4. Flowchart of the impact-set computation.

# 4. UML Design

## *4.1 Class diagram*

Figure 5 shows the classes of the module. StaticAnalyzer is a facade: the other modules call only its analyze method and never use the worker classes directly. This keeps the module's interface small and lets any worker be replaced. The facade owns the scanner, parser, graph builders and analyzers (composition, filled diamond). SymbolTable is used by both the call-graph builder and the matcher, which is the reason resolution logic lives in one place and is not repeated. OracleDetector is deliberately unconnected to the others.

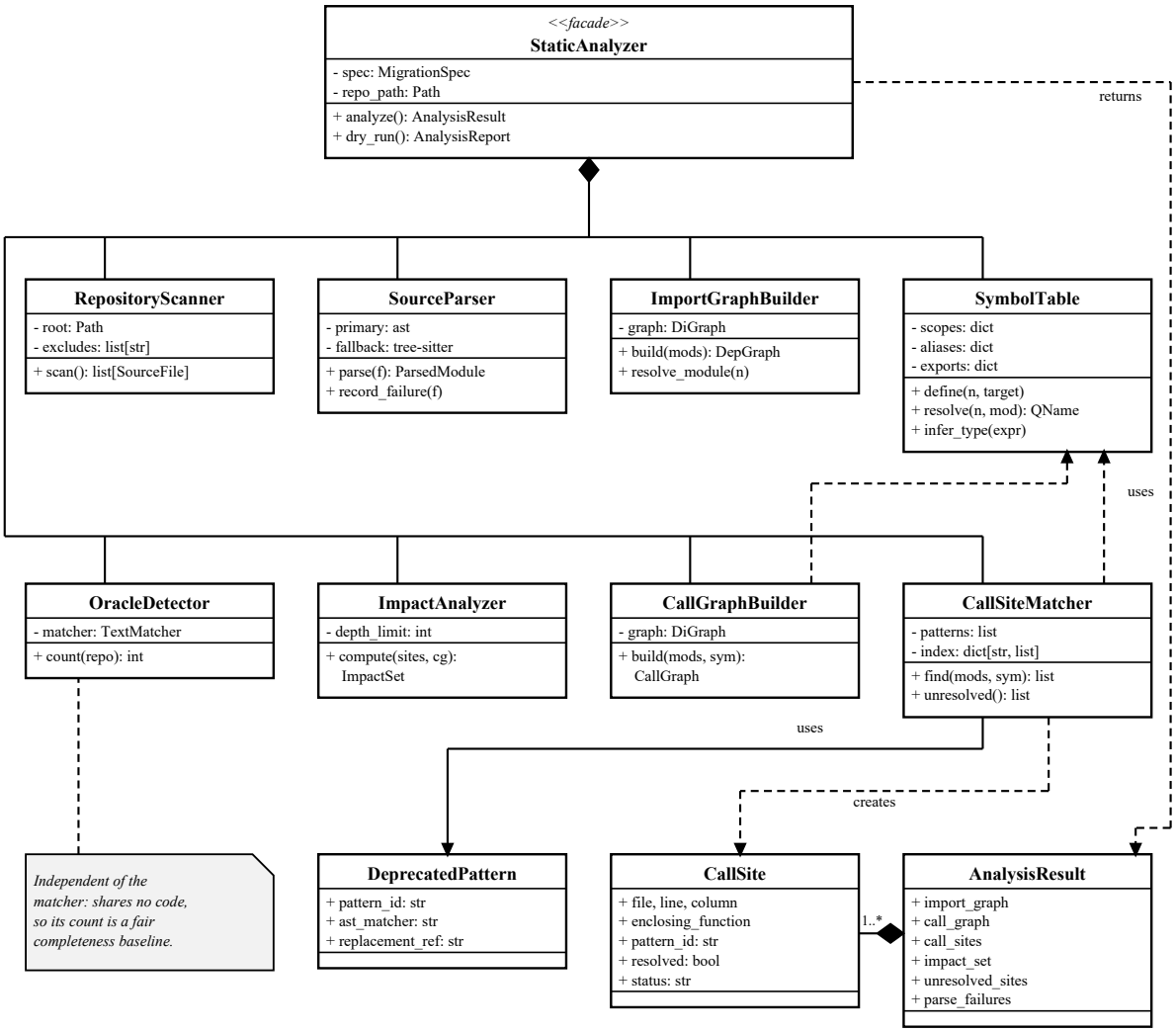



Fig. 5. UML class diagram of Module 2.

| Class | Role | Key design decision |
|---|---|---|
| StaticAnalyzer | Facade and entry point | Runs the steps in the order of Fig. 1 and assembles the AnalysisResult |
| SourceParser | Parse files | Primary and fallback parsers behind one parse() method, so callers do not know which was used |
| SymbolTable | Name resolution | Follows alias chains with a visited set and a depth limit so that cyclic re-exports terminate |
| CallSiteMatcher | Find deprecated calls | Indexes patterns by their final attribute name so most call nodes are rejected in constant time |
| ImpactAnalyzer | Transitive callers | Walk is bounded by depth_limit from the run configuration |
| OracleDetector | Independent baseline | Uses text matching and not the symbol table, so its count does not share the matcher's blind spots |

Table 3. Class responsibilities and design decisions

### *4.2 Sequence diagram*

Figure 6 shows the messages exchanged during one analyze call. The two loop fragments mark the repeated parts: parsing runs once per source file, and name resolution runs once per call node. Every worker returns its result to the StaticAnalyzer, which is the only object that talks to all of them; the workers talk to each other only through SymbolTable.

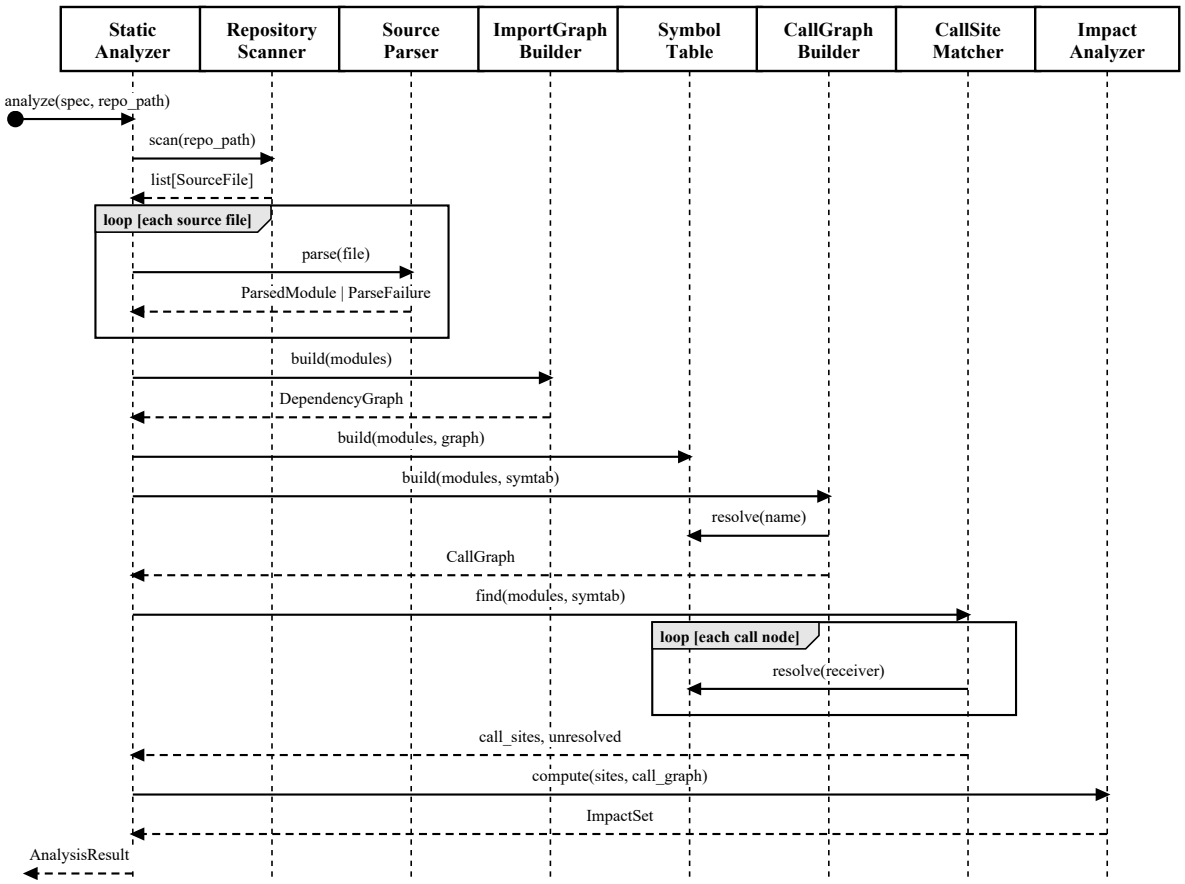



Fig. 6. UML sequence diagram of one analyze() call.

## 5. Data Flow Diagrams

### *5.1 Level 0: context of Module 2*

Figure 7 treats the module as one process. It receives the migration specification from Module 1 and the source files from the repository, and it sends results to three consumers.

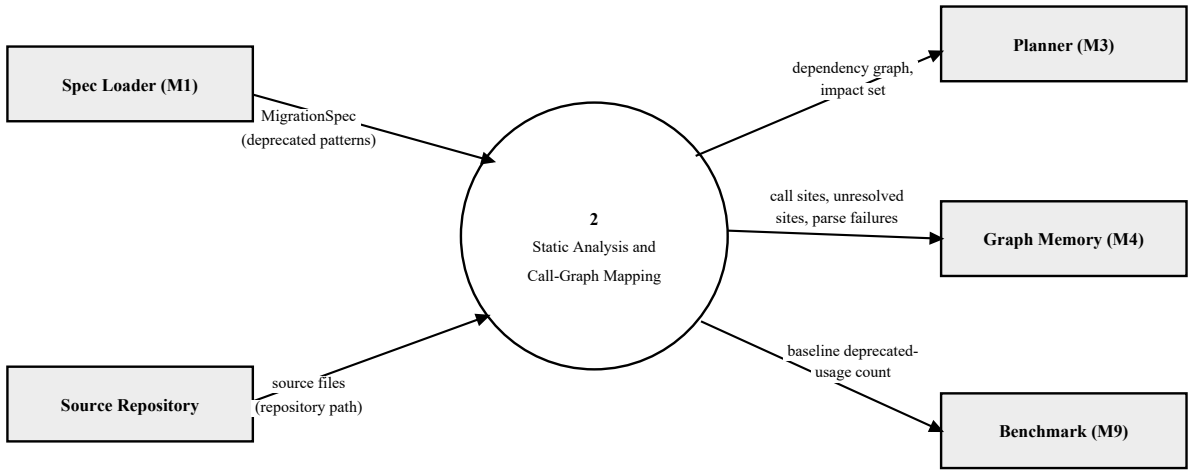



Fig. 7. Data flow diagram of Module 2, level 0.

### *5.2 Level 1: decomposition*

Figure 8 divides the module into its seven sub-components. Flows are numbered F1 to F11 and described in Table 4. The six sub-components 2.1 to 2.6 form a chain, as in the flowchart, and 2.7, the oracle, runs beside it with its own input and output. All intermediate data is held in one data store, the analysis workspace D1, which the sub-components read from and write to. The oracle also reads the pattern names from the specification; this flow is omitted from the figure for clarity.

| Flow | From | To | Data |
|---|---|---|---|
| F1 | Source Repository | 2.1 Scan | Repository path and exclude rules |
| F2 | 2.1 Scan | 2.2 Parse | List of source files |
| F3 | 2.2 Parse | 2.3 Index | Parsed modules (ASTs) and parse failures |
| F4 | 2.3 Index | 2.4 Match | Symbol table and import graph |
| F5 | Spec Loader (M1) | 2.4 Match | Deprecated patterns |
| F6 | 2.4 Match | 2.5 Graph | Call sites and unresolved sites |
| F7 | 2.5 Graph | 2.6 Impact | Call graph |
| F8 | 2.6 Impact | Planner (M3) | Dependency graph and impact set |
| F9 | 2.6 Impact | Memory (M4) | Call sites, unresolved sites and parse failures |
| F10 | Source Repository | 2.7 Oracle | Repository path |
| F11 | 2.7 Oracle | Benchmark (M9) | Baseline deprecated-usage count |

Table 4. Data flows of the level 1 diagram

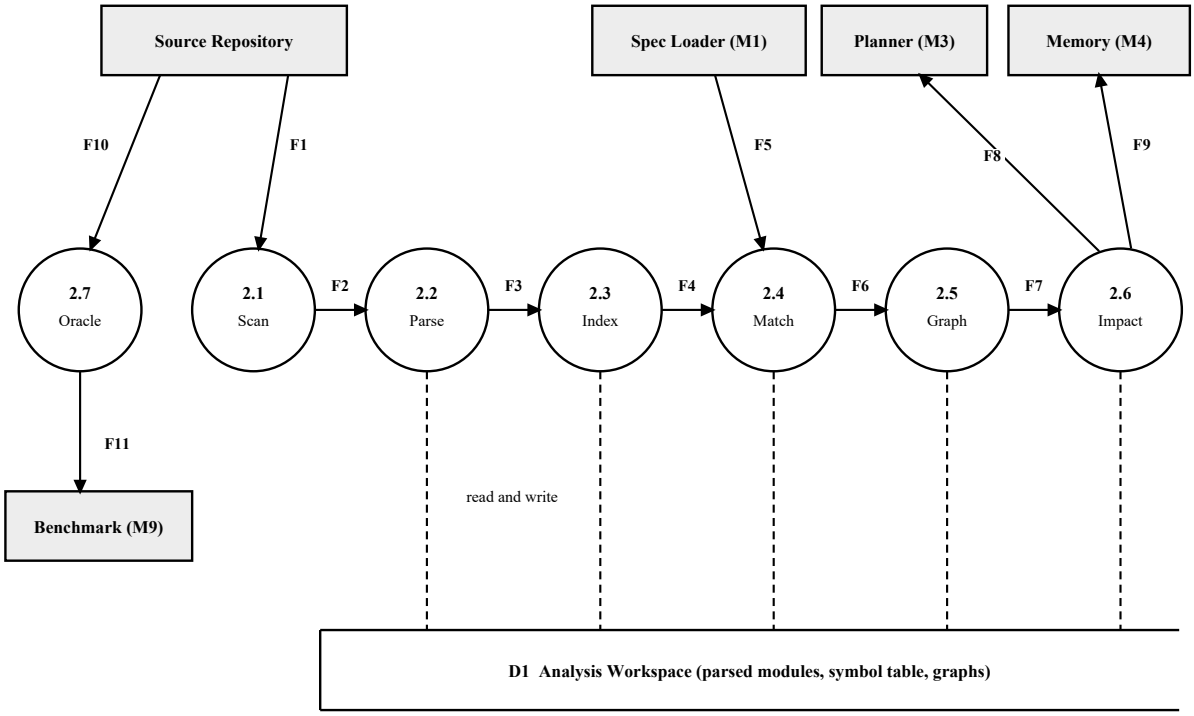



Fig. 8. Data flow diagram of Module 2, level 1.

# 6. Data Structures and Algorithms

## *6.1 Data structures*

| Structure | Representation | Contents |
|---|---|---|
| ParsedModule | Record | File path, syntax tree, source hash, list of imports, top-level definitions |
| DependencyGraph | Directed graph (NetworkX) | Node is a file path; edge from importer to imported file, with the imported symbols and import kind |
| SymbolTable | Dictionary of dictionaries | Per module, maps a local name to a qualified name; also stores alias entries, __all__ lists and the inferred type of local variables |
| CallGraph | Directed graph (NetworkX) | Node is a qualified function name; edge from caller to callee, with call locations |
| CallSite | Record | File, line, column, enclosing function, pattern identifier, resolved flag, status |
| UnresolvedSite | Record | File, line, expression text and a reason: dynamic attribute, dynamic call, unknown receiver or unresolved star import |
| ImpactSet | Sets and a path map | Impacted functions, impacted files, and for each function its depth and nearest call site |

Table 5. Principal data structures

## *6.2 Symbol resolution*

Resolution turns a name used in a module into a qualified name that can be compared with a pattern. It handles a name defined in the same module, an import, an aliased import, a relative import, a star import, and a symbol re-exported through another module.

*resolve(name, module):*

1. head = first segment of name; entry = table[module].get(head)
2. if entry is None: search star imports of module (use __all__ if present, otherwise public names); if still None, return unresolved
3. while entry is an alias or a re-export and entry not in seen and depth < limit: add entry to seen; entry = lookup of its source symbol
4. if head is a local variable: type = infer_type(variable); if type is None return unresolved (unknown receiver)
5. return qualified name = entry + remaining segments of name

For a call such as session.query(User), the receiver session is a local variable, so step 4 applies. The type is taken from the assignment that created it (for example a call to sessionmaker or Session), from a parameter annotation, or, when neither exists, from the Jedi or Pyright inference engine. If all three fail the site is recorded as an unresolved site with the reason unknown receiver, and is not dropped.

### *6.3 Complexity*

| Step | Cost | Note |
|---|---|---|
| Scan and parse | O(total source size) | Dominant cost; files are independent so they can be parsed in parallel |
| Import graph and symbol table | O(files + imports) | One pass over the import statements |
| Call-site matching | O(call nodes) | Pattern index makes the per-node check constant time on average |
| Call graph | O(call nodes) | One edge per resolved call |
| Impact set | O(functions + call edges) | Each function enters the queue at most once |

Table 6. Time complexity of the steps, supporting the scalability requirement NFR4

## 7. Error Handling and Edge Cases

| Situation | Handling |
|---|---|
| Syntax error in a file | Try the fallback parser; if it also fails record a ParseFailure and continue |
| Python 2 source | Read by the fallback parser, since the Python 3 ast module rejects it |
| Circular imports | Allowed in the graph; the planner collapses them into one component, and the symbol resolver's visited set prevents infinite re-export chains |
| Star import | Resolved from __all__ when present, otherwise from public names; unresolved ones are reported |
| Conditional import (try / except ImportError) | Both branches are recorded as possible sources |
| Dynamic dispatch (getattr, eval, importlib) | Recorded as UnresolvedSite with a reason; left to the verification loop |
| Method defined in a parent class | Resolved through the class hierarchy before matching |
| Vendored or generated code | Excluded by the scanner's exclude rules |
| Very large file | Parsed with a time limit; on timeout recorded as a ParseFailure |
| No parsable file | Whole analysis stops with an error (Fig. 1) |

Table 7. Error handling and edge cases

## 8. Test Design

Each design decision above has at least one test case. The tests use small synthetic repositories in which the expected call sites are known in advance.

| ID | Scenario | Expected result |
|---|---|---|
| T1 | Direct call to a deprecated API | One resolved CallSite with correct file and line |
| T2 | Call through an aliased import (import x as y) | Site found despite the alias |
| T3 | Call through a re-export in another module | Site found; re-export chain followed |
| T4 | Method call on a variable created by a factory | Receiver type inferred; site found |
| T5 | Call through getattr | UnresolvedSite with reason dynamic attribute |
| T6 | File with a syntax error | ParseFailure recorded; other files analysed |
| T7 | Python 2 file | Parsed by the fallback parser |
| T8 | Two modules that import each other | Both in the graph; analysis terminates |
| T9 | Recursive function containing a call site | Impact walk terminates; function visited once |
| T10 | Repository with no parsable file | Error reported; no result returned |
| T11 | Oracle count compared with matcher count on a clean case | Counts equal |

Table 8. Test cases for Module 2

## 9. Conclusion

This document has described Module 2 through four flowcharts, a UML class diagram and sequence diagram, and DFDs at two levels. The module is built as a fixed sequence of small, separately testable steps, with unresolved and failed cases recorded and not hidden, and with an independent oracle to check its completeness. Implementation will follow the structure of the class diagram, beginning with the parser and symbol table, since every other class depends on them.